

Explainable ResNet-50 for Brain Tumor MRI Classification

Toan Vo

1 Problem formulation

The real-world problem is the need to support early and accurate identification of brain tumors from MRI scans. This can be framed as an image classification prediction task: given a brain MRI image, the model predicts one of four labels: glioma, meningioma, pituitary tumor, or no tumor. The Kaggle dataset contains MRI images already organized into these four diagnostic categories, making it suitable for supervised machine learning and deep learning classification. The goal is not to replace clinicians, but to build a model that can assist with screening, prioritization, and decision support.

The explanation benefits several groups. Radiologists and clinicians benefit because an interpretable model can highlight why an MRI was classified as a tumor type or as no tumor, helping them judge whether the model’s reasoning is clinically plausible. Patients may also benefit indirectly, since better decision support could reduce delays in diagnosis and treatment planning. For researchers or developers, explanations guide choices about model improvement, such as whether the classifier is learning tumor-related visual features or irrelevant artifacts. In practice, the prediction could help guide follow-up imaging, specialist referral, biopsy consideration, or treatment planning, while still requiring confirmation by medical professionals.

The data was accessed from the public Kaggle dataset “Brain Tumor MRI Dataset” by Masoud Nickparvar [1]. The images were loaded from the dataset folders, with labels assigned according to the folder names. Preliminary preprocessing was applied using PyTorch transforms. For training images, each scan was resized to 224×224 , randomly horizontally flipped with probability 0.5, randomly rotated by up to 10 degrees, converted to a tensor, and normalized using the default mean and standard deviation for pretrained ResNet-50 weights. For evaluation images, the same resizing, tensor conversion, and normalization were applied, but random augmentation was excluded to ensure consistent validation and test results.

2 Target model

The target model used in this project is a ResNet-50 convolutional neural network pretrained on ImageNet and fine-tuned for the brain tumor MRI classification task. The model begins with an initial convolutional layer, batch normalization, ReLU activation, and max pooling, followed by four main residual stages: layer1, layer2, layer3, and layer4. These stages contain bottleneck residual blocks made of 1×1 , 3×3 , and 1×1 convolutional layers. After the convolutional feature extraction stages, the model uses adaptive average pooling to convert the final feature maps into a fixed-length feature vector. The final classification layer is a fully connected linear layer with 2048 input features and 4 output features, where the four outputs correspond to the four MRI classes: glioma, meningioma, no tumor, and pituitary tumor.

```
[1]: import copy
from pathlib import Path
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.models import resnet50, ResNet50_Weights
from sklearn.metrics import classification_report, confusion_matrix
from torchvision.transforms import ToPILImage
import copy
from scipy.stats import spearmanr
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from captum.attr import IntegratedGradients, Saliency
```

c:\Users\gaume\anaconda3\envs\captum-env\Lib\site-packages\tqdm\auto.py:21:
TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

```
[2]: DATA_DIR = Path("dataset")
BATCH_SIZE = 32
NUM_EPOCHS = 20
LR = 1e-4
NUM_WORKERS = 4
MODEL_OUT = "resnet50_brain_tumor.pt"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
[3]: # -----
# Transforms
# -----
weights = ResNet50_Weights.DEFAULT

train_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=weights.transforms().mean,
        std=weights.transforms().std
    ),
])

eval_tfms = transforms.Compose([
```

```

        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=weights.transforms().mean,
            std=weights.transforms().std
        ),
    ])

```

```

[4]: # -----
# Data
# -----
train_ds = datasets.ImageFolder(DATA_DIR / "Training", transform=train_tfms)
val_ds = datasets.ImageFolder(DATA_DIR / "Validation", transform=eval_tfms)
test_ds = datasets.ImageFolder(DATA_DIR / "Testing", transform=eval_tfms)

class_names = train_ds.classes
num_classes = len(class_names)

train_loader = DataLoader(
    train_ds, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=NUM_WORKERS, pin_memory=True
)

val_loader = DataLoader(
    val_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, pin_memory=True
)

test_loader = DataLoader(
    test_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, pin_memory=True
)

```

```

[5]: # -----
# Model
# -----
model = resnet50(weights=weights)

# Replace final ImageNet classifier with tumor-class classifier
in_features = model.fc.in_features
model.fc = nn.Linear(in_features, num_classes)

model = model.to(device)

```

```

[6]: # -----
# Loss, optimizer, scheduler
# -----

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)

```

```

[7]: # -----
# Training helpers
# -----
def run_epoch(model, loader, phase):
    is_train = phase == "train"

    if is_train:
        model.train()
    else:
        model.eval()

    running_loss = 0.0
    running_correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad(set_to_none=True)

        with torch.set_grad_enabled(is_train):
            outputs = model(images)
            loss = criterion(outputs, labels)
            preds = outputs.argmax(dim=1)

            if is_train:
                loss.backward()
                optimizer.step()

        batch_size = images.size(0)
        running_loss += loss.item() * batch_size
        running_correct += (preds == labels).sum().item()
        total += batch_size

    epoch_loss = running_loss / total
    epoch_acc = running_correct / total

```

```

        return epoch_loss, epoch_acc

def train_model(model):
    best_model_wts = copy.deepcopy(model.state_dict())
    best_val_acc = 0.0

    for epoch in range(NUM_EPOCHS):
        train_loss, train_acc = run_epoch(model, train_loader, "train")
        val_loss, val_acc = run_epoch(model, val_loader, "val")

        scheduler.step(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_wts = copy.deepcopy(model.state_dict())

        print(
            f"Epoch {epoch + 1:02d}/{NUM_EPOCHS} | "
            f"train loss {train_loss:.4f}, train acc {train_acc:.4f} | "
            f"val loss {val_loss:.4f}, val acc {val_acc:.4f} | "
        )

    model.load_state_dict(best_model_wts)
    torch.save(
        {
            "model_state_dict": model.state_dict(),
            "class_names": class_names,
            "num_classes": num_classes,
        },
        MODEL_OUT
    )

    print(f"\nBest validation accuracy: {best_val_acc:.4f}")
    print(f"Saved model to: {MODEL_OUT}")

    return model

```

```

[8]: # -----
# Evaluation
# -----
def evaluate(model, loader):
    model.eval()

    all_preds = []
    all_labels = []

```

```

with torch.no_grad():
    for images, labels in loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(dim=1).cpu()

        all_preds.extend(preds.tolist())
        all_labels.extend(labels.tolist())

print("\nClassification report:")
print(classification_report(all_labels, all_preds, target_names=class_names))

print("\nConfusion matrix:")
print(confusion_matrix(all_labels, all_preds))

```

```

[9]: # # Training code (IGNORE)
# print(f"Using device: {device}")
# print(f"Classes: {class_names}")

# model = train_model(model)
# evaluate(model, test_loader)

```

```

[10]: checkpoint = torch.load(MODEL_OUT, map_location=device)
model.load_state_dict(checkpoint["model_state_dict"])
model = model.to(device)
evaluate(model, test_loader)

```

Classification report:

	precision	recall	f1-score	support
glioma	1.00	0.82	0.90	400
meningioma	0.91	0.98	0.94	400
notumor	0.91	1.00	0.95	400
pituitary	0.99	0.99	0.99	400
accuracy			0.95	1600
macro avg	0.95	0.95	0.95	1600
weighted avg	0.95	0.95	0.95	1600

Confusion matrix:

```

[[329  36  35   0]
 [  1 392   3   4]
 [  0   1 399   0]
 [  0   2   0 398]]

```

3 Explanation

The method used here is Saliency maps. This method used here supports local explanations. Both gradient saliency and integrated gradients explain the model's prediction for one specific MRI image at a time, rather than summarizing the model's behavior across the whole dataset. In this case, the visualization shows which pixels or image regions most influenced the ResNet-50 model's classification decision for this particular scan. A global explanation would require aggregating results across many images, for example by comparing average saliency patterns for glioma, meningioma, pituitary tumor, and no tumor cases.

```
[11]: # Get class of test image
      test_image, test_label = test_ds[0]
      test_label
```

```
[11]: 0
```

```
[12]: def tensor_to_image(tensor, weights):
      # Handles [1, 3, H, W] or [3, H, W]
      if tensor.ndim == 4:
          tensor = tensor[0]

      tensor = tensor.detach().cpu()

      mean = torch.tensor(weights.transforms().mean).view(3, 1, 1)
      std = torch.tensor(weights.transforms().std).view(3, 1, 1)

      tensor = tensor * std + mean
      tensor = tensor.clamp(0, 1)

      return ToPILImage()(tensor)

      test_image_vis = tensor_to_image(test_image, weights)
```

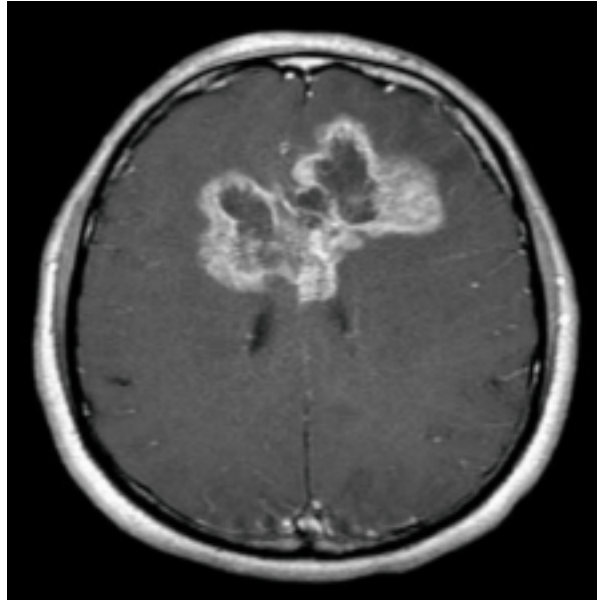
```
[13]: test_image = test_image.unsqueeze(0) if test_image.ndim == 3 else test_image
      test_image = test_image.clone().detach().to(device).requires_grad_(True)

      saliency = Saliency(model)
      attr_grad = saliency.attribute(test_image, target=test_label)

      ig = IntegratedGradients(model)
      attr_ig = ig.attribute(test_image, target=test_label, n_steps=50)
```

```
[14]: test_image_vis
```

```
[14]:
```



```
[15]: def attribution_to_array(tensor):
    arr = tensor.squeeze().detach().cpu().numpy()

    if arr.ndim == 3:
        arr = arr.mean(axis=0)

    return np.abs(arr)

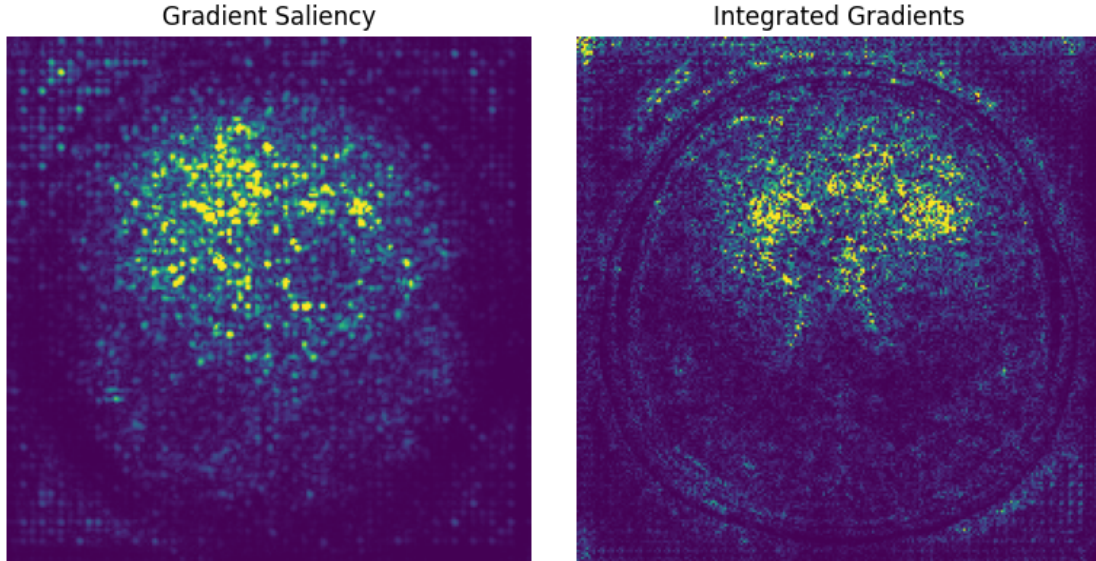
attr_grad_arr = attribution_to_array(attr_grad)
attr_ig_arr = attribution_to_array(attr_ig)

fig, axes = plt.subplots(1, 2, figsize=(8, 4))

methods = [
    ("Gradient Saliency", attr_grad_arr),
    ("Integrated Gradients", attr_ig_arr),
]

for ax, (title, arr) in zip(axes, methods):
    vmax = np.percentile(arr, 99)
    im = ax.imshow(arr, vmin=0, vmax=vmax)
    ax.set_title(title)
    ax.axis("off")

plt.tight_layout()
plt.show()
```

In this example, the gradient saliency map appears noisy and places strong importance on the top left regions of the brain, where there weren't any tumor present. This suggests that plain gradient saliency may be sensitive to local pixel-level changes and may highlight distracting or less clinically meaningful features. In contrast, the integrated gradients map shows stronger focus around the tumor region. Integrated gradients provides a clearer and more clinically plausible explanation than raw gradient saliency in this case.

One way this output could be misread is to assume that every bright region is definitely tumor tissue or that the explanation proves the model is correct. These maps show model sensitivity, not a medical segmentation mask. Bright areas indicate where the model's prediction was most affected, but they may also reflect edges, contrast, noise, or preprocessing artifacts. This is visible in the Integrated Gradients map, where some highlighted areas appear around the surrounding boundary of the brain rather than only on the tumor region. To avoid this misreading, the explanation should be examined alongside the original MRI image, the predicted label, the true label, and ideally expert tumor annotations. It is also important to compare explanations across multiple scans rather than drawing conclusions from a single image.

4 Hyperparameter check

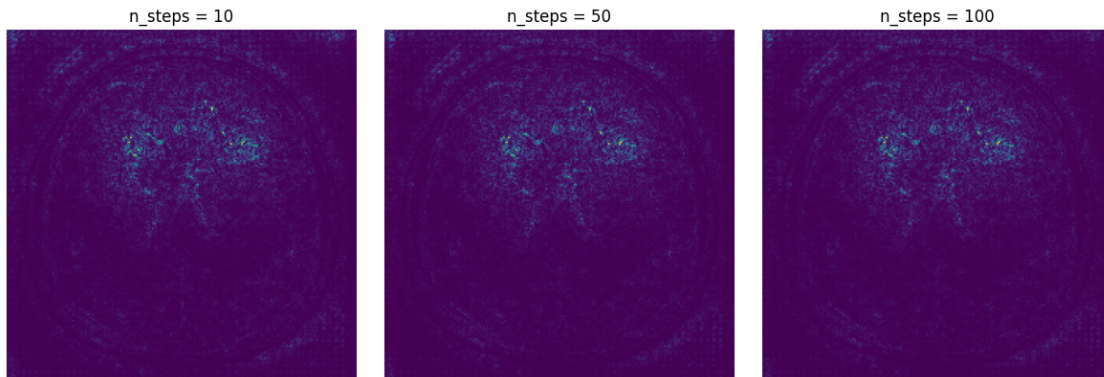
Integrated Gradients has an important hyperparameter called `n_steps`. This controls the number of interpolation points used between a baseline input and the actual MRI image. A low value such as `n_steps = 10` gives a faster but rougher approximation of feature importance, while a higher value such as `n_steps = 200` gives a more stable explanation at greater computational cost. I tested the sensitivity of the Integrated Gradients output by comparing attribution maps with `n_steps = 10`, 50, and 100.

The result shows that the explanation are relatively stable, since the highlighted tumor region remains similar across these values.

```
[16]: steps = [10, 50, 100]
      arrs = []
      for n_steps in steps:
          attr_ig = ig.attribute(test_image, target=test_label, n_steps=n_steps)
          attr_ig_arr = attribution_to_array(attr_ig)
          arrs.append(attr_ig_arr)

[17]: fig, ax = plt.subplots(1, len(steps), figsize=(12, 4))
      # vmax = np.percentile(attr_ig_arr, 99)
      for i, (attr_ig_arr, n_steps) in enumerate(zip(arrs, steps)):
          ax[i].imshow(attr_ig_arr, vmin=attr_ig_arr.min(), vmax=attr_ig_arr.max())
          ax[i].set_title(f"n_steps = {n_steps}")
          ax[i].axis("off")

      plt.tight_layout()
      plt.show()
```



5 Detailed Check

```
[18]: def attr_flat(attr):
      return attr.squeeze().abs().mean(dim=0).detach().cpu().numpy().ravel()

      cascade_layers = ["fc", "layer4", "layer3", "layer2", "layer1", "conv1"]
      layer_order = ["Original"] + [f"+ {l}" for l in cascade_layers]

      model_rand = copy.deepcopy(model)
      model_rand.eval()

      x = test_image.unsqueeze(0) if test_image.ndim == 3 else test_image
      x = x.clone().detach().to(device).requires_grad_(True)

      attrs = {"Original": (attr_ig, attr_grad)}
```

```

scores = {"Original": (1.0, 1.0)}
orig_flat_ig = attr_flat(attr_ig)
orig_flat_grad = attr_flat(attr_grad)

for layer_name in cascade_layers:
    for param in model_rand.get_submodule(layer_name).parameters():
        torch.nn.init.normal_(param.data, std=0.1)

    key = f"+ {layer_name}"
    attrs[key] = (IntegratedGradients(model_rand).attribute(x,
→target=test_label), Saliency(model_rand).attribute(x, target=test_label))

    scores[key] = (spearmanr(orig_flat_ig, attr_flat(attrs[key][0])).correlation,
                    spearmanr(orig_flat_grad, attr_flat(attrs[key][1])).
→correlation)

```

```

[19]: randomization_df = pd.DataFrame({
        "layer": layer_order,
        "spearman_ig": [scores[layer][0] for layer in layer_order],
        "spearman_grad": [scores[layer][1] for layer in layer_order]
    })

randomization_df

```

```

[19]:
   layer  spearman_ig  spearman_grad
0  Original      1.000000      1.000000
1    + fc      0.574957      0.623582
2  + layer4      0.416203      0.444027
3  + layer3      0.398210      0.399418
4  + layer2      0.294019      0.322315
5  + layer1      0.271107      0.281303
6  + conv1      0.245938      0.258712

```

```

[20]: fig, axes = plt.subplots(2, len(layer_order), figsize=(3 * len(layer_order), 6))

# Plot visualizations. First row is IG, second row is Gradient Saliency
for i, layer in enumerate(layer_order):
    ig_attr, grad_attr = attrs[layer]

    ig_arr = attribution_to_array(ig_attr)
    grad_arr = attribution_to_array(grad_attr)

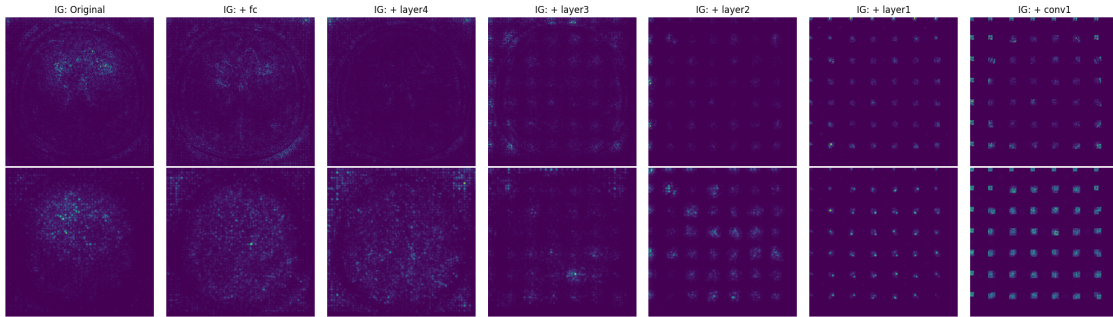
    axes[0, i].imshow(ig_arr, vmin=ig_arr.min(), vmax=ig_arr.max())
    axes[0, i].set_title(f"IG: {layer}")
    axes[0, i].axis("off")

    axes[1, i].imshow(grad_arr, vmin=grad_arr.min(), vmax=grad_arr.max())

```

```
axes[1, i].axis("off")

plt.tight_layout()
plt.show()
```



The results show that both Integrated Gradients and Gradient Saliency deteriorate as more of the model is randomized. For Integrated Gradients, the Spearman correlation drops from 1.00 for the original model to 0.529 after randomizing the fully connected layer, then continues decreasing to 0.377, 0.395, 0.279, 0.239, and finally 0.227 after randomizing through conv1. For Gradient Saliency, the correlation drops from 1.00 to 0.621 after the fully connected layer, then to 0.454, 0.342, 0.356, 0.223, and 0.232. This downward trend means the attribution maps become less similar to the original explanation as more learned layers are destroyed.

This suggests that the explanation methods are at least partly responding to the trained ResNet-50 model rather than only showing generic image edges or preprocessing patterns. Overall, the saliency methods pass the model randomization check because the explanations deteriorate when the model's learned parameters are randomized, although Integrated Gradients provides the more convincing explanation for this MRI example.